



JavaScript Essentials And Advanced

➤ What is JavaScript? Explain the role of JavaScript in web development.

- ✓ JavaScript is a high-level, interpreted, dynamic programming language as well as scripting language.
- ✓ It's used to create interactive and dynamic web pages, updating webpages content without reloading.
- ✓ JS handles features such as animation, events and user interactions.
- ✓ It works together HTML & CSS where HTML provides structure while CSS provides styling and JS adds functionality and behaviour to the webpage.
- ✓ It can run directly to the browser and is supported by all the Morden to the browser.

➤ How is JavaScript different from other programming languages like Python or Java?

JS	PYTHON	JAVA
Used for create webpages	Used for AI ,data science ,automation	Used for enterprise, android applications
It dynamically types language	it dynamically types language	It statically types language
Runs directly in browser	Runs using Python interpreter	Run on JVM (Java Virtual Machine)

➤ Discuss the use of <script> tag in HTML. How can you link an external JavaScript File to an HTML document?

- ✓ The <script> tag in HTML used to add JavaScript to a Webpages.
- ✓ It's using webpages more integrative and dynamic.
- ✓ JavaScript can be written inside <script> tag and linking using an external file.
- ✓ The src attribute is used to connect an external JavaScript files.
- ✓ E.X : <script src = "javascript.js"> </script>

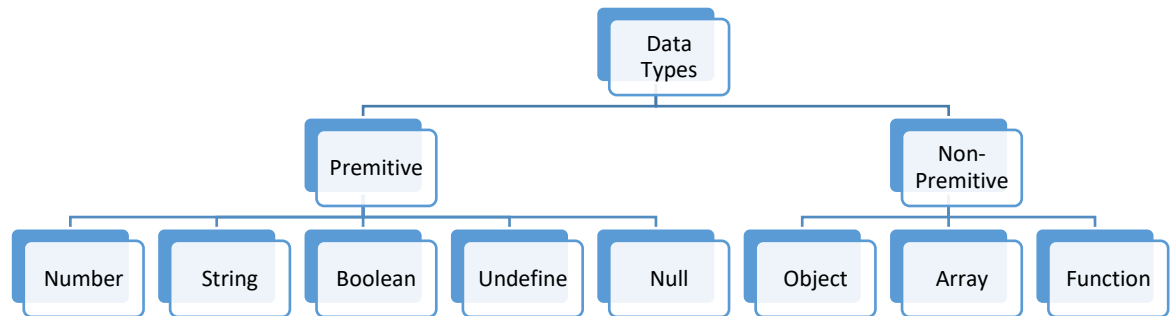
➤ What are variables in JavaScript? How do you declare a variable using var, let, and const?

- ✓ Variables in JavaScript are used to store data values.
- ✓ They help store information such as numbers, text, and other data types.
- ✓ JavaScript provides var, let, and const keywords to declare variables.
- ✓ EX : var name = "JS" // can be redeclared and reassign
Let a = 20 // can be reassign but can't be redeclared
Const pi = 3.14 // can't be both

➤ **Explain the different data types in JavaScript. Provide examples for each.**

Definition:

“In JS, Datatypes are attribute that tell engine how to store, Interpreter and perform operation on a values”



- ✓ **Number:** Used to store numbers like int, float and so on.
EX: let num = 20;
- ✓ **String:** Used to store collection of characters, written in quotes.
EX: let string_example = "Hello";
- ✓ **Boolean:** it return only 2 values True or False.
EX: let isStudent = True;
- ✓ **Undefined:** it called when variable declared but no values assigned.
EX: let un;
- ✓ **Null:** it used to represent an empty value or empty object reference that means assign value intentionally but no contains data.
EX: let data = null;
- ✓ **Object:** it used to store data in key – values pairs.
EX: const person_data = {
 "Name": "Megha Patel",
 "Role": "Tutor"
 }
- ✓ **Array:** it's collection of multiple data, store values in a single variable.
EX: let FruitList = ["Green Apple", "Banana", "Cherry"]
- ✓ **Function:** A block of code used to perform a specific task.
EX: function test () {
 Alert ("Welcome to JS Webpage Test..!");
 }

➤ **What is the difference between undefined and null in JavaScript?**

Undefined	Null
A variable is declared but no value is assigned.	A variable is assigned an empty value intentionally.
Default value given by JavaScript.	Value assigned by the programmer.
Type is undefined.	Type is object.
EX :	EX :

Let un; Console.log(un) // Undefined	Let obj = null; Console.log(obj) // null
---	---

➤ **What are the different types of operators in JavaScript? Explain with examples.**

Definition:

“Operators is a special type of symbol used to perform specific actions on values and variable that called operands”

- **Arithmetic operators** : Used to perform mathematical calculations

Symbol	Description	Example
+	Addition	10 + 5
-	Subtraction	10 – 5
*	Multiplication	10 * 5
/	Division	10 / 5
%	Modulus	10 % 5
**	Exponent	10 ** 5

- **Assignment operators** : Used to assign values to variables

Symbol	Description	Example
=	Assign value	A = 10
+=	Add and assign	A += 10
-=	Subtract and assign	A -= 89
*=	Multiply and assign	A *= 5
/=	Divide and assign	A /= 45

- **Comparison operators** : Used to compare two values

Symbol	Description	Example
==	Equal to	5 == “5”
===	Strict equal	5 === “5”
!=	Not equal	5 != 5
>	Greater than	5 > 5
>=	Less than	5 >= 5
<	Greater or equal	5 < 5
<=	Less or equal	5 <= 5

- **Logical operators** : Used to combine conditions

Symbol	Description	Example
&&	AND	True && False
	OR	True False
!	NOT	!true

➤ **What is the difference between == and === in JavaScript?**

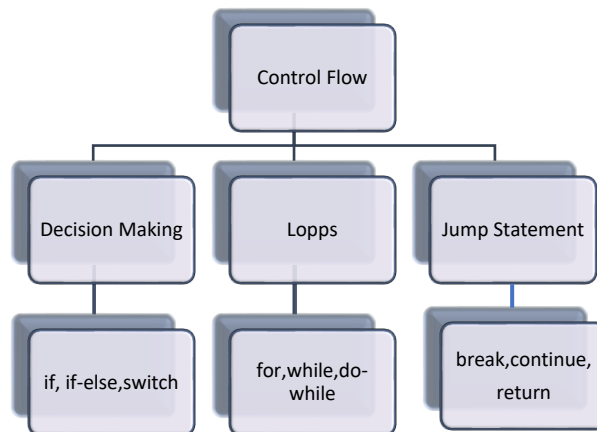
==	===
----	-----

Used to check the value equal or not	Used to strictly check the values equal or not
Performs type conversion if needed	Does not perform type conversion
EX: <code>Console.log(5 == "5") // true</code>	EX: <code>Console.log(5 === 5) // false</code>

➤ **What is control flow in JavaScript? Explain how if-else statements work with an example.**

Definition:

“Control flow refers to order in which code executes based on decisions & conditions, such as if-else statement to run different block of code”



- ✓ It runs code from top – bottom And left – right direction.

IF-ELSE Statement

- ✓ The If-else statement used to check the condition. If first block is false then it jump to second block like
- ✓ If the condition is true then if block execute otherwise it execute the else block.
- ✓ EX:

```

If (let a >= 18) {
    Console.log ("Eligible for voting..!")
} else {
    Console.log ("You are not eligible for voting..1")
}
  
```

➤ **Describe how switch statements work in JavaScript. When should you use a switch statement instead of if-else?**

Definition:

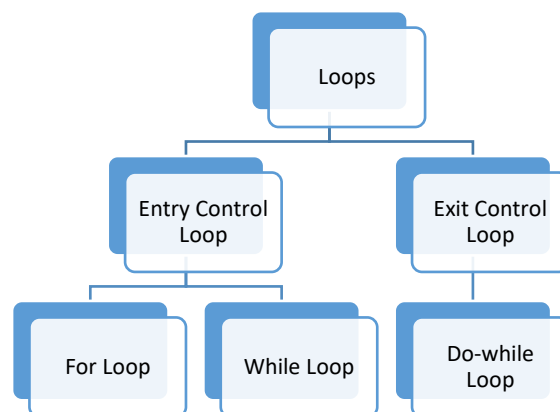
“In JS statement, its control flow tool, used to execute the different block of code based on the value of expression”

- ✓ Switch case statement used to improve code readability, organization, and maintainability.
- ✓ It is easier to maintain than long if-else statements.
- ✓ Commonly used for menus, roles, days, and predefined options.

➤ **Explain the different types of loops in JavaScript (for, while, do-while). Provide a basic example of each.**

Definition :

“Loops are used to perform repetitive tasks by executing a block of code multiple times until the condition is met”



- **For loop**
 - ✓ Used when the number of iterations is known.
 - ✓ It contains initialization, condition, and increment/decrement in one line
 - ✓ EX

```
for(i = 0; i < 5; i++)
```

- **While loop**
 - ✓ Used when the number of iterations is not fixed.
 - ✓ It checks the condition first before execution
 - ✓ EX

```
I = 0;
while(i < 5){ i++;}
```

- **Do-while loop**
 - ✓ Similar to while loop but executes the block at least one time.
 - ✓ Condition is checked after execution
 - ✓ EX

```
do {
  i++;
} while(i < 5)
```

➤ **What is the difference between a while loop and a do-while loop?**

while Loop	do-while Loop
Checks the condition before execution	Checks the condition after execution
May execute zero times if condition is false	Executes at least one time even if condition is false
Entry controlled loop	Exit controlled loop
EX: <pre> I = 0; While(I > 0) { I++; } </pre>	EX <pre> Do{ I++; }while(i>0) </pre>

➤ **What are functions in JavaScript? Explain the syntax for declaring and calling a function.**

- ✓ Functions in JavaScript are reusable blocks of code used to perform a specific task.
- ✓ A function executes only when it is called (invoked).
- ✓ Functions help reduce code repetition and improve code reusability
- ✓ Syntax

```

Function fun_name(){
    // block of code
}
Fun_name() // calling function

```

➤ **What is the difference between a function declaration and a function expression?**

Function Declaration	Function Expression
Function is declared using the function keyword with a name	Function is stored inside a variable
Can be called before declaration	Cannot be called before declaration
Declared independently	Declared as a value/expression
EX <pre> Function greet() { Console.log("hello"); } </pre>	EX <pre> Let greet = function(){ Console.log('hello'); } </pre>

➤ **Discuss the concept of parameters and return values in functions.**

- Parameters
 - ✓ Parameters are variables used in a function definition to receive values from the user.
 - ✓ They make functions flexible and reusable
 - ✓ EX

```

Function add(a,b){ }

```

- Return Values

- ✓ Return values are the results sent back from a function using the return keyword.
- ✓ They allow the function to provide output after execution
- ✓ EX

```
Function add(a,b){
    Return a + b
}
Let res = add(10,20)
Console.log(res)
```

➤ **What is an array in JavaScript? How do you declare and initialize an array?**

- ✓ An array in JavaScript is a special variable used to store multiple values in a single variable.
- ✓ Arrays can store numbers, strings, objects, and other data types.
- ✓ Array values are stored using index numbers starting from 0
- ✓ EX

```
Let name = ["rivu", "denu"]
```

➤ **Explain the methods push (), pop (), shift (), and unsift () used in arrays.**

- Push () : Adds one or more elements to the end of an array
- ✓ EX

```
let arr = [1, 2];
arr.push(3);
```

- Pop () : Removes the last element from an array
- ✓ EX

```
let arr = [1, 2];
arr.pop(3);
```

- Shift () : Removes the first element from an array
- ✓ EX

```
let arr = [1, 2];
arr.shift(3);
```

- Unsift () : Adds one or more elements to the beginning of an array
- ✓ EX

```
let arr = [1, 2];
arr.unsift(3);
```

➤ **What is an object in JavaScript? How are objects different from arrays?**

- ✓ An object in JavaScript is a collection of data stored in key-value pairs.
- ✓ Objects are used to store related properties and methods of an entity.
- ✓ EX

```
Let person = {
```

```
Name: "john",  
Age: 25  
};
```

Object	Array
Stores data in key-value pairs	Stores multiple values in ordered form
Uses property names (keys)	Uses index numbers
Written using { }	Written using []
EX Person.name	EX Colour [0]

➤ **Explain how to access and update object properties using dot notation and bracket notation.**

- dot notation

- ✓ Object properties in JavaScript can be accessed and updated using dot notation and bracket notation
- ✓ Uses a dot (.) followed by the property name
- ✓ EX

```
let person = {  
  name: "John",  
  age: 25  
};  
console.log(person.name);  
person.age = 30;
```

- bracket notation

- ✓ Uses square brackets [] with the property name as a string
- ✓ EX

```
console.log(person["name"]);  
person["age"] = 30;
```

➤ **What are JavaScript events? Explain the role of event listeners.**

- ✓ JavaScript events are actions or activities that occur in the browser, such as clicking a button, pressing a key, moving the mouse, or submitting a form.
- ✓ Events help make webpages interactive and responsive to user actions.
 - Event Listener
 - ✓ Event listeners are used to detect and handle events in JavaScript.
 - ✓ They wait for a specific event to occur and then execute a function.

EX

```
button.addEventListener("click", function() {  
  alert("Button clicked");  
});
```

➤ **How does the `addEventListener()` method work in JavaScript?**

Provide an example.

- ✓ The `addEventListener()` method is used to attach an event handler to an HTML element.
- ✓ It listens for a specific event such as `click`, `mouseover`, or `keydown`, and executes a function when the event occurs.
- ✓ It allows multiple events to be added to the same element without overwriting existing events
- ✓ EX

```
let btn = document.getElementById("myBtn");
btn.addEventListener("click", function() {
  alert("Button clicked");
});
```

➤ **What is the DOM (Document Object Model) in JavaScript? How does JavaScript interact with the DOM?**

- ✓ The DOM (Document Object Model) is a programming interface that represents an HTML document as a tree structure of elements and objects.
- ✓ It allows JavaScript to access, modify, add, or delete HTML elements and content dynamically.
- ✓ JavaScript interacts with the DOM using methods like `getElementById()`, `querySelector()`, and event handlers.
- ✓ Through the DOM, JavaScript can change text, styles, attributes, and respond to user actions without reloading the webpage
- ✓ EX

```
document.getElementById("demo").innerHTML = "Hello";
```

➤ **Explain the methods `getElementById()`, `getElementsByClassName()`, and `querySelector()` used to select elements from the DOM.**

- `getElementById()`
 - ✓ Used to select an element by its unique `id`
 - ✓ EX
- `getElementsByClassName()`
 - ✓ Used to select all elements with the same class name.
 - ✓ Returns an HTML collection.
 - ✓ EX
- `querySelector()`
 - ✓ Used to select the first element that matches a CSS selector
 - ✓ EX

```
document.getElementById("demo");
```

```
document.getElementsByClassName("box");
```

```
document.querySelector(".box");
```

➤ **Explain the `setTimeout ()` and `setInterval ()` functions in JavaScript. How are they used for timing events?**

- `setTimeout()`

- ✓ `setTimeout ()` is used to execute a function only once after a specified time delay

- ✓ EX

```
setTimeout(function() {  
    alert("Hello");  
}, 2000);
```

- `setInterval()`

- ✓ `setInterval ()` is used to repeatedly execute a function after a specified time interval

- ✓ EX

```
setInterval(function() {  
    console.log("Running");  
}, 1000);
```

➤ **Provide an example of how to use `setTimeout ()` to delay an action by 2 seconds.**

- ✓ `setTimeout ()` delays the execution of the function.

- ✓ 2000 represents 2000 milliseconds (2 seconds)

- ✓ EX

```
setTimeout(function() {  
    alert("Action performed after 2 seconds");  
}, 2000);
```

➤ **What is error handling in JavaScript? Explain the `try`, `catch`, and `finally` blocks with an example.**

- ✓ Error handling in JavaScript is used to detect and manage runtime errors so the program does not stop unexpectedly.

- ✓ The `try` block contains code that may cause an error.

- ✓ The `catch` block handles the error if it occurs.

- ✓ The `finally` block always executes whether an error occurs or not

- ✓ EX

```
try {  
    let result = 10 / 0;  
    console.log(result);  
  
} catch(error) {  
    console.log("Error occurred");  
  
} finally {  
    console.log("Execution completed");  
}
```

➤ **Why is error handling important in JavaScript applications?**

- ✓ Error handling is important in JavaScript because it helps prevent the program from crashing due to runtime errors.
- ✓ It improves application stability and user experience by handling errors gracefully.
- ✓ Error handling also helps developers identify, debug, and manage unexpected issues effectively